# W09B · 한 장 지도 · 같은 뉴스, 다른 학습 정답

2026-10-28 · 생성형 인공지능 9주차 2차시 · 『핸즈온 생성형 AI』 6.1–6.3

중간고사 전에는 사전학습 모델에 입력을 주고 결과를 관찰했습니다. 이제는 **입력과 정답으로 손실을 계산하고 모델의 가중치를 실제로 조정**합니다. 지난 실습의 실행 결과나 저장 변수가 없어도 새 Colab 사본에서 시작할 수 있습니다.

### 같은 뉴스, 서로 다른 학습 정답

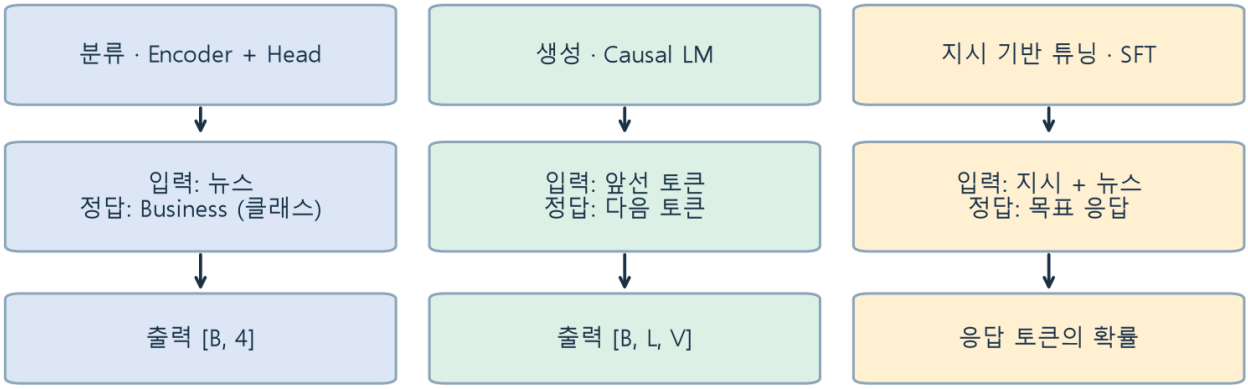



그림 읽기: 각 열에서 **입력 → 정답 → 출력의 형태**를 따라가세요. 오른쪽의 지시 데이터 구성과 실제 SFT 학습은 구분합니다. 출처: 주교재 6.1–6.3의 개념을 바탕으로 새로 설계한 수업 도식.

| 작업 | 모델에 주는 것 | 손실에서 맞히려는 것 | 확인할 결과 |
|---|---|---|---|
| 뉴스 분류 | 뉴스의 토큰·마스크 | World/Sports/Business/Sci/Tech 중 정답 ID | 네 클래스 점수·혼동행렬 |
| 뉴스 이어쓰기 | 앞선 토큰 | 바로 다음 토큰 | 검증 NLL·생성 문장 |
| 지시 기반 튜닝 | 지시문·입력 | 목표 응답의 토큰 | 보지 않은 지시에서의 이행·내용·형식 |

## 읽고 실행하는 순서

1. 분류 노트를 읽으며 **Colab A**의 토큰·분류 헤드·짧은 학습을 관찰합니다.
2. 생성 노트와 **Colab B**에서 다음 토큰 정답과 학습 전후를 비교합니다.
3. 지시 사항 노트에서 지시 데이터와 평가 사례를 설계합니다.
4. 마지막 웹앱에서 입력과 함수의 연결을 확인하고 실험 기록지에 예상·관찰·설명을 남깁니다.

각 Colab은 **Drive에 새 사본을 저장하고 첫 셀부터** 실행합니다. CPU 기본, API 키·개인 파일 업로드·유료 업그레이드는 필수가 아닙니다. 최초 모델 파일은 A 약269 MB, B 약541 MB이며 데이터와 의존성 다운로드가 추가됩니다. 받은 파일은 캐시에서 재사용합니다.

## 실습 바로가기

- Colab A · 뉴스 분류 파인튜닝
- Colab B · 뉴스 생성과 지시 데이터
- 함수·클래스·메소드 설명과 실제 정의

패키지는 `luna-genai 0.1.15`, 설치 기준은 `2026-fall-w09b`입니다. 이미 이전 버전을 import했다면 런타임을 재시작하고 첫 셀부터 실행합니다. 웹앱 공유 URL은 실행 중인 Colab 런타임에 의존합니다.

## 이번 수업의 확인 기준

실행 전에는 결과를 예상하고, 실행 후에는 **무엇을 정답으로 삼았는지·무엇이 학습됐는지·비교 조건을 어떻게 고정했는지·이 결과로 어디까지 설명할 수 있는지**를 적습니다. 학습이 실행됐다는 사실과 성능이 좋아졌다는 판단은 별도로 확인합니다.

짧은 학습은 작동 원리를 관찰하는 실험입니다. 교재의 장시간 학습 결과나 상용 서비스의 품질을 재현했다고 해석하지 않습니다. 별도 성적 반영 과제나 새로운 제출 기한은 없습니다.

# W09B · 6.1 · 뉴스 분류 모델은 무엇을 학습하는가

주교재 6.1.1–6.1.7 · 238–257쪽 · Colab A와 함께 읽는 노트

## 1. 프롬프트로 분류하기에서 학습하는 분류기로

이전 W02B 실습은 문맥 뒤의 `positive`와 `negative` 후보 토큰 확률을 비교했습니다. 예시나 지시를 프롬프트에 추가해도 그 과정에는 `backward()`나 가중치 갱신이 없었습니다.

이번에는 뉴스와 주제 라벨을 짝지어 모델을 학습합니다. 사전학습 인코더는 언어의 표현을 제공하고, 새 분류용 층은 그 표현을 네 클래스 점수로 바꿉니다. 전체 파인튜닝에서는 **인코더와 분류용 층 모두** 손실의 기울기를 받아 갱신됩니다.

파인튜닝은 이미 배운 언어 표현을 이용하면서 새로운 목적에 맞게 모수를 조정하는 과정입니다. 기본 모델을 불러오는 것만으로 AG News 분류 학습이 끝나는 것은 아닙니다.

**A1 · 필수 · 먼저 예상하기:** 분류 결과가 달라졌을 때, 입력을 바꾼 효과인지 가중치를 학습한 효과인지 구별하려면 무엇을 고정하고 어떤 값을 확인해야 할까요? Colab A의 A1에 자신의 예상과 이유를 남기세요.

## 2. 데이터부터 선택하기 — 6.1.1

교재는 AG News를 사용합니다. 공식 학습 데이터는120,000개, test는7,600개이며 다음 네 주제의 영어 뉴스로 구성됩니다.

| 라벨 ID | 원래 클래스 이름 | 한국어로 읽기 |
|---|---|---|
| 0 | World | 세계 뉴스 |
| 1 | Sports | 스포츠 |
| 2 | Business | 비즈니스·경제 |
| 3 | Sci/Tech | 과학·기술 |

한 행의 핵심은 `text: str`, `label: int`입니다. 클래스 이름의 나열 순서가 곧 출력 열의 의미입니다. 2를 두 번째 클래스라고 읽으면 잘못된 대응이 됩니다. Python의 ID는0부터 시작합니다.

수업 로더 `load_news()`는 고정된 데이터 revision을 내려받고 캐시를 재사용합니다. 기본 작은 실험은 클래스마다 학습64개·검증32개·최종 test32개를 선택합니다. 검증은 공식 train에서 분리하며, 대소문자·공백을 정규화한 중복 문서가 서로 다른 분할에 들어가지 않게 합니다. 이 검사는 의미가 비슷한 기사나 바꿔 쓴 문장까지 모두 찾아내는 것은 아닙니다.

학습 데이터는 가중치 계산에, validation은 설정·학습 상태 점검에 사용합니다. test를 반복해서 보며 설정을 고르면 최종 평가의 독립성이 약해집니다. 테스트 문서를 학습에 넣는 것뿐 아니라, 같은 기사의 복사본이 검증에 섞이는 것도 주의해야 합니다.

### 파인튜닝의 일곱 단계와 평가 경계

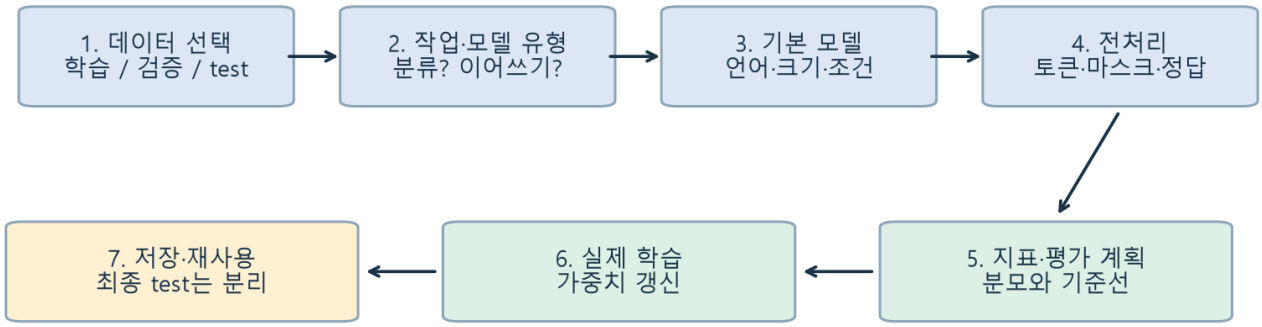



그림 읽기: 지표는 학습이 끝난 뒤 갑자기 정하지 않습니다. **무엇을 잘해야 하는지와 평가 분할을 먼저 정하고** 학습합니다. 출처: 주교재239쪽 그림6-1의 일곱 단계 개념을 수업 흐름에 맞게 재구성.

## 3. 모델 유형과 기본 모델 — 6.1.2–6.1.3

| 유형 | 기본적인 정보 흐름 | 적합한 작업의 예 |
|---|---|---|
| 인코더 | 입력 문맥의 표현을 계산 | 문장 분류·개체명 인식 |
| 디코더 기반 causal LM | 앞선 토큰으로 다음 토큰을 예측 | 문장 이어쓰기 |
| 인코더–디코더 | 입력을 인코딩하고 별도 출력 시퀀스를 생성 | 번역·요약 |

작업과 구조는 일대일의 절대 규칙이 아닙니다. 생성 모델도 지시로 분류할 수 있습니다. 이번에는 **고정된 네 주제 분류**를 위해 인코더와 분류 헤드를 선택하여 학습 과정과 평가를 명확히 살펴봅니다.

기본 모델은 교재와 같은 `distilbert/distilbert-base-uncased`입니다. 실제 로드한 분류 모델은 약6,696만 파라미터, 인코더6층, 표현 차원768입니다. 영어 소문자 처리를 전제로 하므로 영어 기사에 맞습니다. 이 선택을 한국어 뉴스의 성능 보장으로 확장하면 안 됩니다.

모델 선택에는 언어·입력 길이·모델 크기·메모리·학습 시간·평가 결과·이용 조건이 함께 필요합니다. 이번 작은 CPU 실험에서 좋은 모델이 다른 과제에서도 최선이라고 결론 내리지는 않습니다.

## 4. 토큰·마스크·분류 헤드 — 6.1.4

### 사전학습 표현에서 네 클래스의 점수로

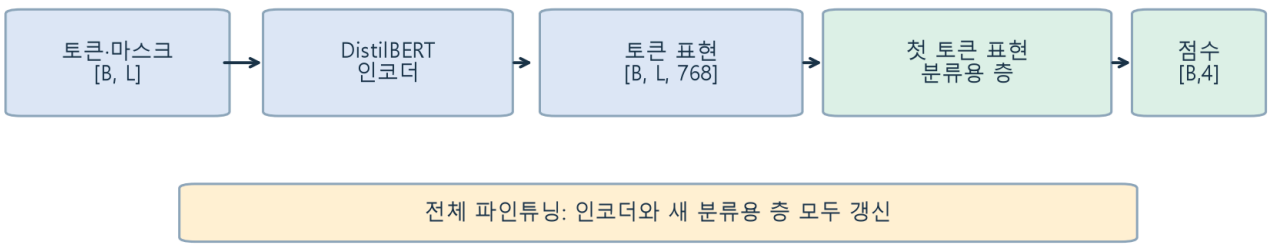


그림 읽기: 인코더가 만드는 토큰별 표현과, 최종 문장 분류 점수는 다릅니다. 출처: 주교재248쪽 그림6-2의 인코더·분류 헤드 관계를 실제 DistilBERT 차원에 맞춰 새로 도식화. DistilBERT의 분류용 층은 첫 토큰 표현에 추가 변환·활성화·dropout·선형 층을 적용합니다.

배치 크기를 $B$, 배치 안에서 맞춘 토큰 길이를 $L$, 표현 차원을 $H$라 하면 다음처럼 읽을 수 있습니다.

| 값 | 크기 | 의미 |
|---|---|---|
| input_ids | $[B, L]$ | 어휘 사전의 정수 토큰 ID |
| attention_mask | $[B, L]$ | 실제 토큰1, padding0 |
| 마지막 은닉 표현 | $[B, L, H]$, 여기서는 $H = 768$ | 각 입력 토큰의 문맥 표현 |
| 분류 logits | $[B, 4]$ | 네 클래스의 정규화 전 점수 |
| 분류 labels | $[B]$ | 문장마다 하나의 정답 클래스 |

토큰 ID는 단어의 점수나 클래스 번호가 아닙니다. ID 숫자가 크다고 의미가 강해지는 것도 아닙니다. logits는 확률이 아니며, 클래스 축에 softmax를 적용해야 확률처럼 합이1인 값으로 바뀝니다.

## 길이 맞추기와 loss에서 제외할 위치

| 설명용 input_ids | 101 | 314 | 202 | 102 | 0 | 0 |
|---|---|---|---|---|---|---|
| attention_mask | 1 | 1 | 1 | 1 | 0 | 0 |
| 문장 분류 labels | 2 (Business) | | | | | |

ID는 구조 설명용입니다. 실제 토큰·ID와 truncation 결과는 Colab에서 확인합니다.

**padding**은 서로 다른 길이를 배치의 최대 길이에 맞추는 작업입니다. **truncation**은 길이 제한을 넘는 내용을 자르는 작업입니다. 마스크는 원래 잘린 뒷부분을 복구하지 않습니다. 수업에서는 계산량을 줄이기 위해 최대 64토큰을 사용하므로 기사 뒷부분의 유용한 정보가 사라질 수 있습니다. 길이512를 지원하는 모델이라고 항상 512로 학습해야 하는 것은 아닙니다.

**A3 · 필수 · 관찰하기:** 길이가 다른 두 입력의 input_ids, attention_mask, labels를 확인하고 padding과 truncation이 각각 어떤 위치에 영향을 주는지 설명하세요. 힌트는0의 위치와 실제 토큰 수입니다. 자기 점검: 문장당 정답 개수와 토큰당 입력 개수를 구별했나요?

# 5. 손실에서 가중치 갱신까지 — 6.1.6

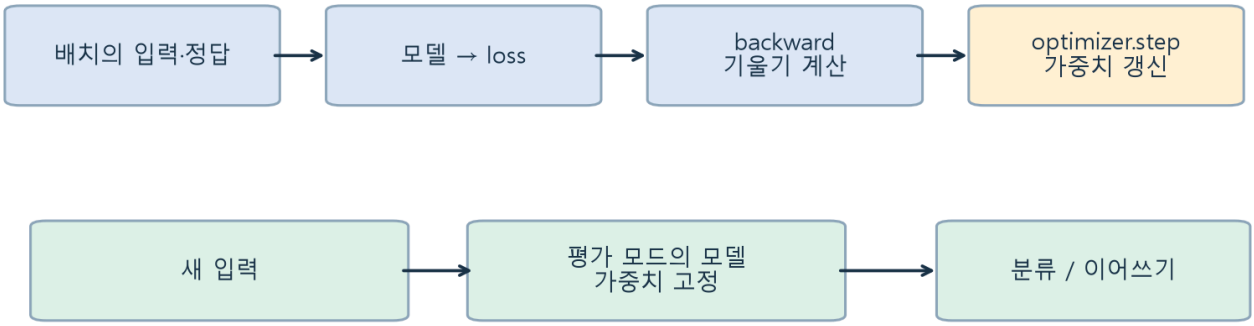
분류의 목표는 정답 클래스에 높은 확률을 주는 것입니다. 문장 $i$의 정답 ID를 $y_i$, 모델의 클래스 확률을 $p_\theta(k \mid x_i)$라 하면 평균 cross-entropy는 다음과 같습니다.

$$\mathcal{L}_{\text{cls}} = -\frac{1}{B}\sum_{i=1}^{B} \log p_\theta(y_i \mid x_i).$$

설명용 예에서 정답 확률이0.8이면 그 문장의 손실은 $-\log(0.8) \approx 0.223$입니다. 정답 확률이0.2이면 손실은 약1.609로 커집니다. 이 수계산은 실제 AG News 평가 수치가 아닙니다.

**A2 · 확장 · 작은 계산:** Colab의 다른 확률 예에서 정답 라벨을 바꾸고 손실을 계산하세요. 목표는 같은 출력 점수라도 정답에 따라 손실이 달라짐을 확인하는 것입니다.

### 학습의 한 단계와 추론을 구별하기




학습의 기본 흐름은 다음과 같습니다.

```
optimizer.zero_grad(set_to_none=True)
output = model(**batch)
loss = output.loss
loss.backward()
optimizer.step()
```

`model(**batch)`는 사전의 이름을 함수 인자로 풀어 전달하는 Python 문법입니다. `labels`를 함께 주면 이 작업에 맞는 손실도 계산합니다. `backward()`는 기울기를 계산하고, `optimizer.step()`이 그 기울기로 가중치를 갱신합니다. 이 두 작업은 같은 의미가 아닙니다. 다음 배치 전에 기울기를 초기화하는 이유는 PyTorch가 기본적으로 기울기를 누적하기 때문입니다.

`model.train()`과 `model.eval()`은 dropout 등의 동작 모드를 바꿉니다. `eval()`만 호출해 기울기 기록까지 꺼지는 것은 아니므로 추론에는 `torch.inference_mode()`도 사용합니다. `eval()`이 모델을 이전 가중치로 되돌리는 것도 아닙니다.

교재의 `Trainer`는 이러한 학습·평가 루프를 감싸는 도구입니다. 이번 최소 실험은 공통 `FineTuningLab.fit()`의 짧은 루프를 읽어 중간 과정을 추적합니다. 실제 설정은 AdamW·고정 학습률·weight decay0.01·기울기 norm clipping1.0입니다. 이 최소 루프에는 학습률 스케줄러가 없으며, 교재의 cosine 스케줄과 구분합니다.

**A4 · 필수 · 디버깅:** `backward()`만 호출한 코드와 `optimizer.step()`만 호출한 코드를 점검하세요. 실제 학습 후에는 분류용 층과 인코더의 지정된 파라미터 일부가 변했는지 확인합니다. 힌트: 기울기 계산과 모수 갱신을 분리해 설명하세요.

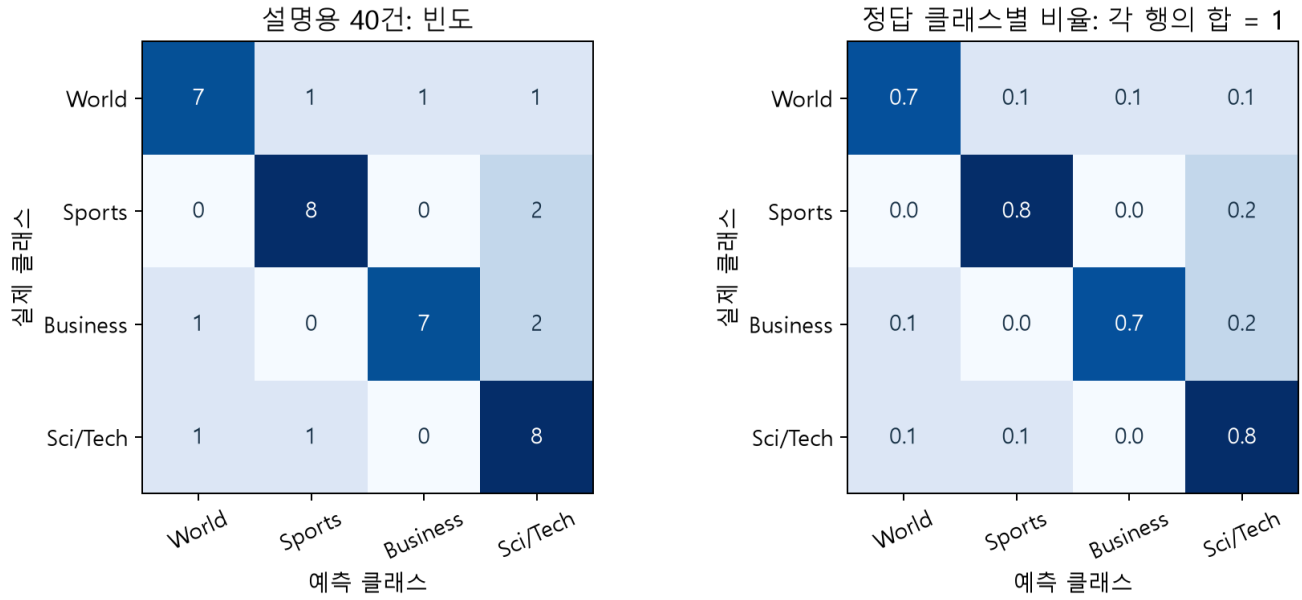
처음 모델을 불러올 때 분류용 층의 가중치가 새로 초기화되었다는 메시지는 예상되는 상황입니다. 기존 사전 학습 목적의 출력 층과 이번 분류 헤드가 다르기 때문입니다. 이를 학습을 마친 분류기라는 증거로 읽으면 안 됩니다.

## 6. 무엇으로 평가할까 — 6.1.5

정확도는 전체 예측 중 맞힌 비율입니다. 특정 클래스의 precision은 그 클래스로 **예측한 것** 중 정답 비율, recall은 실제 그 클래스인 것 중 **찾아낸 비율**입니다.

$$\mathrm{Precision}_k = \frac{TP_k}{TP_k + FP_k}, \qquad \mathrm{Recall}_k = \frac{TP_k}{TP_k + FN_k}, \qquad F1_k = \frac{2TP_k}{2TP_k + FP_k + FN_k}.$$

분모가0인 클래스의 지표는 이번 구현에서0으로 정합니다. macro F1은 클래스별 F1을 같은 비중으로 평균내고, 교재 코드의 weighted F1은 실제 클래스별 표본 수를 가중치로 사용합니다. 클래스가 불균형하면 정확도와 weighted F1이 높아도 소수 클래스의 recall이 낮을 수 있습니다.




행은 실제 클래스, 열은 예측 클래스입니다. 오른쪽은 각 행의 표본 수로 나눈 값이므로 같은 실제 클래스가 어디로 분류됐는지 읽습니다. 그림은 직접 만든 설명용40건이며 실제 학습 성능표가 아닙니다.

**A5 · 확장 · 비교하기:** 다수 클래스만 예측한 작은 예의 정확도·macro F1·weighted F1을 계산하고 소수 클래스에서 놓친 것을 설명하세요. 자기 점검: precision과 recall의 분모를 구별했나요?

## 7. 전체 학습과 표현 추출, 그리고 실용성 — 6.1.6–6.1.7

인코더를 고정하고 분류용 층만 학습하면 학습 대상이 줄어듭니다. 인코더를 고정한 표현에 별도 로지스틱 회귀 같은 분류기를 붙이는 방식도 가능합니다. 이들은 전체 파인튜닝과 갱신되는 파라미터가 다릅니다. 어떤 방식이 더 나은지는 데이터 양·계산 자원·목표와 실제 평가에 따라 확인합니다.

**A6 · 확장 · 가설 세우기:** 인코더를 고정하면 학습 가능한 파라미터 수, 시간, 표현 변화가 어떻게 달라질지 먼저 예상하세요. 수를 확인한 결과와 성능 효과를 검증한 결과를 구별합니다.

작은 분류기는 빠른 대량 처리, 일관된 라벨 공간, 제한된 자원에서 유용할 수 있습니다. 교재는 문서 품질 필터링 등에도 이러한 모델을 연결합니다. 다만 뉴스 분류 정확도는 다른 도메인의 품질 판정이나 편향 없는 판단을 보장하지 않습니다. 지표와 함께 실제 오류 문장, 사용 목적, 데이터 범위, 처리 시간을 확인해야 합니다.

## 실습 API와 출처

`load_news()`의 분할 목록을 `FineTuningLab.batch/fit/evaluate`에 전달하고, 웹앱은 같은 객체의 `predict()`를 호출합니다. `fit`은 현재 모델을 계속 갱신하므로 설정 비교에는 같은 seed의 새 객체를 준비합니다. 실제 정의와 입력·출력 설명을 먼저 읽으세요.

- 주교재: 『핸즈온 생성형 AI』 6.1,238–257쪽. 수업용 수계산·작은 표본·도식·학습 루프·macro F1 비교는 추가 설명입니다.
- Hugging Face 분류 안내
- AG News 데이터 카드
- DistilBERT 모델 카드

# W09B · 6.2 · 클래스 라벨 없이 무엇을 정답으로 학습할까

주교재 6.2.1–6.2.2 · 257–268쪽 · Colab B와 함께 읽는 노트

## 1. 같은 뉴스가 다른 학습 문제가 된다

뉴스 분류에서는 기사 하나에 Business 같은 정답 클래스가 있었습니다. 뉴스 이어쓰기에서는 앞선 토큰을 보고 **바로 다음 토큰**을 맞힙니다. 원래 문장 자체가 다음 토큰의 정답을 제공하므로 사람이 모든 위치의 정답을 따로 붙일 필요는 없습니다.

따라서 데이터의 주제 label 열을 제거하는 것과 학습용 정답이 사라지는 것은 다릅니다. 생성 모델을 학습할 때 쓰는 labels는 토큰화한 문장에서 구성하는 별도의 텐서입니다.

**B1 · 확장 · 먼저 예상하기:** Business 기사만 고른 뒤 원래 클래스 열을 제거하면, 모델이 맞히는 정답은 어디서 올까요? 다른 뉴스 주제를 선택했을 때 바뀌는 입력 분포와 그대로인 학습목표를 구별하세요.

## 2. 생성 모델 선택의 기준 — 6.2.1

이번 모델은 교재와 같은 HuggingFaceTB/SmolLM-135M입니다. 실제 로드한 모델은 약1억3,452만 파라미터, 은닉 차원576, 어휘 크기49,152인 causal LM입니다. 사전학습 base 모델이며 대화 지시 이행을 보장하는 챗봇으로 소개하지 않습니다.

| 선택 기준 | 확인할 내용 | 이번 실습에 주는 의미 |
|---|---|---|
| 모델 크기 | 파라미터 수·가중치 dtype·학습 상태 메모리 | 작은 모델로 실제 갱신을 관찰 |
| 학습 데이터와 언어 | 데이터 출처·언어·정제·편향 | 영어 뉴스 실험이며 한국어 품질을 보장하지 않음 |
| 컨텍스트 길이 | 모델 한도와 실제 학습 길이 | 모델 한도2048, CPU 필수 실험은32토큰 |
| 라이선스·사용 조건 | 모델 카드·코드·데이터 조건을 각각 확인 | 접근 가능성과 원자료 재배포를 구별 |
| 평가 결과 | 무엇을 어떤 조건에서 평가했는지 | 순위 하나로 뉴스 생성 품질을 확정하지 않음 |

가중치의 메모리는 대략 **파라미터 수 × 원소당 바이트 수**로 계산할 수 있습니다. 그러나 학습에는 기울기·optimizer 상태·중간 활성값도 필요합니다. 모델 파일541 MB를 받았다는 사실이 학습 메모리도541 MB라는 뜻은 아닙니다. 실제 장치와 실행 시간을 기록해야 합니다.

교재의 모델·벤치마크 표는 책이 쓰인 시점의 비교 자료입니다. 이번 선택은 작은 모델의 학습 원리를 관찰하기 위한 것이며 최신 최고 성능 순위를 뜻하지 않습니다.

## 3. Business 기사와 입력·정답 구성 — 6.2.2

Colab B는 고정된 AG News에서 Business 문서를 선택합니다. 공식 train 안에서 학습·validation을 분리하고 official test는 최종 점검에만 사용합니다. 문서마다 `id`와 `text`를 남기고 원래 주제 라벨은 학습 입력에서 제거합니다.

토큰화 뒤 배치에는 다음 값이 있습니다.

| 값 | 크기 | 의미 |
|---|---|---|
| `input_ids` | $[B, L]$ | 본문의 토큰 ID |
| `attention_mask` | $[B, L]$ | 실제 토큰1·padding0 |
| 생성 학습 `labels` | $[B, L]$ | 입력에서 복사한 정답 토큰; padding은-100 |
| 생성 `logits` | $[B, L, V]$ | 위치마다 전체 어휘에 대한 점수, $V = 49{,}152$ |

분류의 출력 크기 $[B, 4]$와 비교해 보세요. 생성 모델은 문장 하나에 주제 네 개의 점수만 내놓지 않습니다. **각 토큰 위치마다 다음 토큰 후보 전체의 점수**를 계산합니다.

### 다음 토큰 정렬: labels를 두 번 이동시키지 않기

| 입력 | The | company | grew | EOS | PAD |
|---|---|---|---|---|---|
| labels (복사) | The | company | grew | EOS | -100 |
| 이 위치에서 맞힐 다음 정답 | company | grew | EOS | 제외 | 없음 |

단어를 토큰으로 단순화한 예시. 실제 EOS는 정답으로 유지하고 padding 위치만 제외합니다.

그림은 단어를 토큰으로 단순화한 설명입니다. 실제 토큰 경계와 ID는 `target_table(batch, tokenizer)`의 출력으로 확인합니다. 수업용 신규 도식이며 실제 모델의 토큰 배열이라고 표시하지 않습니다.

## 4. 다음 토큰 정렬과 두 가지 마스크

causal LM은 현재 위치에서 미래의 정답 토큰을 미리 볼 수 없도록 **causal attention**을 사용합니다. 여기에 입력 길이를 맞추기 위한 **padding mask**가 추가됩니다. 학습 손실에서 무시할 위치를 지정하는 `labels=-100`은 다시 별도의 역할입니다.

수업 수집기는 `labels=input_ids.clone()`으로 정답을 복사한 뒤 `attention_mask==0`인 위치만-100으로 바꿉니다. 이 단계에서는 정답 배열을 옆으로 이동시키지 않습니다. 모델의 손실 계산에서 **앞 위치의 logits와 바로 다음 위치의 labels**를 한 번 맞춥니다. 수집기에서도 이동시키고 모델에서도 이동시키면 한 칸을 더 건너뛰는 잘못된 문제가 됩니다.

SmolLM의 기본 토크나이저에는 padding 토큰이 없어 이번 실습은 EOS를 PAD로도 사용합니다. 하지만 같은 ID라고 같은 역할인 것은 아닙니다. 문장 끝의 **실제 EOS**는 학습 정답으로 남기고, 길이를 맞추려고 붙인 **padding 위치**만 손실에서 제외합니다. `input_ids == pad_token_id`인 모든 곳을 무조건 지우면 실제 EOS 정답도 지워질 수 있습니다.

**B2 · 필수 · 디버깅:** Colab에서 길이가 다른 두 문장의 마스크를 확인하고, EOS와 PAD의 ID가 같은 상황에서 손실 제외 조건을 고치세요. 힌트: 토큰의 숫자만 보지 말고 그 위치가 실제 입력인지 확인합니다. 자기 점검: 마지막 실제 EOS와 뒤의 padding을 구별했나요?

## 5. 생성 학습의 손실과 평가 분모

토큰 시퀀스를 $x_1, \ldots, x_L$라 하면 다음 토큰 예측의 손실은 다음과 같이 표현할 수 있습니다. $\mathcal{T}$는 padding 등 무시할 위치를 제외한 유효한 다음 토큰 위치의 집합입니다.

$$\mathcal{L}_{\mathrm{LM}} = -\frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} \log p_\theta(x_t \mid x_{<t}).$$

문장 분류는 문장마다 하나의 정답을, 생성 학습은 여러 토큰 위치의 정답을 사용합니다. `labels`의 크기와 손실의 분모가 달라지는 이유입니다. 두 작업 모두 cross-entropy를 사용할 수 있지만 같은 숫자의 손실을 같은 성능으로 비교하지는 않습니다.

길이가 다른 배치의 NLL을 모을 때는 각 배치의 유효 정답 토큰 수를 가중치로 사용합니다.

$$\mathrm{NLL}_{\mathrm{all}} = \frac{\sum_b n_b \, \mathrm{NLL}_b}{\sum_b n_b}, \qquad \mathrm{PPL} = \exp(\mathrm{NLL}_{\mathrm{all}}).$$

PPL은 사용한 토크나이저·분할·입력 길이·손실 계산 조건과 함께 읽습니다. 다른 어휘 사전을 쓰는 모델의 PPL을 그대로 나란히 놓고 순위를 정하면 안 됩니다. 낮은 NLL이 사실성이나 지시 이행을 직접 보장하는 것도 아닙니다.

**B3 · 확장 · 작은 계산:** 유효 토큰 수가 다른 두 배치의 평균 손실을 합쳐 보세요. 단순히 두 평균을 다시 평균내는 것과 어떤 차이가 생기나요? Colab의 `target_tokens`와 손실 분모를 함께 확인합니다.

## 6. 짧은 실제 학습을 관찰하기

생성 실습도 `forward` → `loss` → `backward` → `optimizer.step`으로 가중치를 갱신합니다. 모델 전체를 처음부터 학습하는 과정이 아니라 사전학습 SmolLM을 작은 뉴스 표본으로 더 학습하는 과정입니다.

실습에서 먼저 학습 전 validation NLL과 생성 문장을 기록합니다. 이어서 `fit()`을 실행하고 **같은 validation 문서·같은 프롬프트·같은 생성 설정**으로 비교합니다. 프롬프트도 바꾸고 학습도 하면 두 변화가 섞입니다.

학습률은 한 번의 갱신 크기에 영향을 주고, weight decay는 모수에 대한 규제를 더합니다. learning-rate scheduler는 단계에 따라 학습률을 바꾸는 도구입니다. 이번 최소 루프는 고정 학습률을 사용하며, 모델의 입력 문맥 길이·배치 크기·학습 횟수도 실행 기록에 남깁니다.

짧은 학습에서는 학습 손실이 매 단계 단조롭게 줄지 않을 수 있습니다. 매번 다른 문장과 dropout을 사용하기 때문입니다. 손실 그래프 한 점만으로 실패나 성공을 단정하지 말고, 같은 조건의 평가와 실제 파라미터 갱신을 함께 보세요.

모델이 Business 문체에 더 잘 맞춰져도 모든 뉴스 사실을 정확히 기억하거나 지시를 잘 따르게 되는 것은 아닙니다. 생성물은 실제 뉴스 보도로 사용하지 않습니다. 수업의 영어 시작 문장 `The company announced`는 "그 회사는 발표했다"라는 의미이며, 모델이 어떤 이어쓰기를 선호하는지 관찰하기 위한 예입니다.

## 7. 학습과 생성 설정의 효과를 분리하기

`temperature=0`은 이번 API에서 greedy 생성입니다. temperature가 양수이면 `top_p=0.9`를 사용해 샘플링합니다. 샘플링의 seed는 비교 조건을 재현하기 위한 기록값입니다. 입력·모델·정밀도·실행 환경도 함께 고정해야 하며 서로 다른 장치의 결과가 항상 바이트 단위로 같다고 보장하지 않습니다.

**B4 · 확장 · 통제 비교:** 학습 전후를 비교할 때와 temperature만 바꿔 비교할 때, 각각 무엇을 고정해야 할까요? 결과를 본 뒤 예상 문장을 수정하고, 문체 변화·반복·내용 오류를 구별해 적습니다.

생성은 여러 번 모델을 호출하여 문맥 뒤에 토큰을 붙이는 추론 과정입니다. 생성 길이만 늘리는 동안 `optimizer.step()`으로 가중치를 학습하는 것은 아닙니다. 이 점은 중간고사 전 샘플링에서 가중치를 고정했던 것과 연결됩니다.

## 8. 코드와 평가의 연결

`FineTuningLab('causal')`는 실제 모델을 한 번 준비합니다. `batch()`는 입력·마스크·정답 텐서를 만들고, `fit()`은 그 객체의 가중치를 갱신합니다. `evaluate()`는 유효 토큰 수로 집계한 NLL을 반환하며, `generate()`는 프롬프트를 제외한 새 텍스트와 실제 설정을 반환합니다. `save()`/`load()`는 로컬 저장·복원이며 Hub 업로드가 아닙니다.

모든 함수의 인자·반환값·부작용과 실제 정의는 W09B API 안내에 있습니다. 코드가 오류를 내면 마지막 오류를 읽고 원인을 해결합니다. 실패한 모델 출력을 가짜 결과로 바꾸어 학습 성공으로 표시하지 않습니다.

## 출처와 수업의 추가 설명

- 주교재: 『핸즈온 생성형 AI』 6.2,257–268쪽. 표본 축소·가중치 변화 관찰·EOS/PAD 회귀 검사·평가 분모 비교는 수업에서 추가한 설명입니다.
- Hugging Face causal LM 안내
- Data collator의 labels·padding 계약
- SmolLM-135M 모델 카드

# W09B · 6.3 · 지시를 쓰는 것과 지시를 따르도록 학습하는 것

주교재 6.3 "지시 사항" · 268–273쪽 상부 · Colab B의 지시 데이터·웹앱 활동

## 1. 작업마다 모델을 다시 준비해야 할까

하나의 뉴스 분류기를 만들었다고 그 모델이 요약·번역·대화까지 모두 같은 방식으로 처리하는 것은 아닙니다. 교재는 이러한 상황에서 작업별 파인튜닝, 어댑터, 프롬프팅, 지도 기반 파인튜닝 등의 접근을 비교합니다.

작업별 모델은 목적과 평가를 명확히 정하기 쉽습니다. 대신 작업이 늘어나면 여러 모델의 학습·저장·운영을 관리해야 합니다. 프롬프팅은 이미 준비한 모델에 지시나 예시를 입력하는 방법입니다. 새 입력에 반응하는 추론이며 그 요청만으로 모델 가중치를 추가 학습하지는 않습니다.

지시 기반 튜닝은 **지시와 입력에 대해 어떤 응답을 해야 하는지 보여 주는 학습 예제**로 모델을 조정합니다. SFT는 supervised fine-tuning, 즉 지도 기반 파인튜닝을 뜻합니다. 지시 기반 튜닝은 이러한 학습을 지시문·응답 데이터에 적용하는 맥락에서 이해할 수 있습니다.

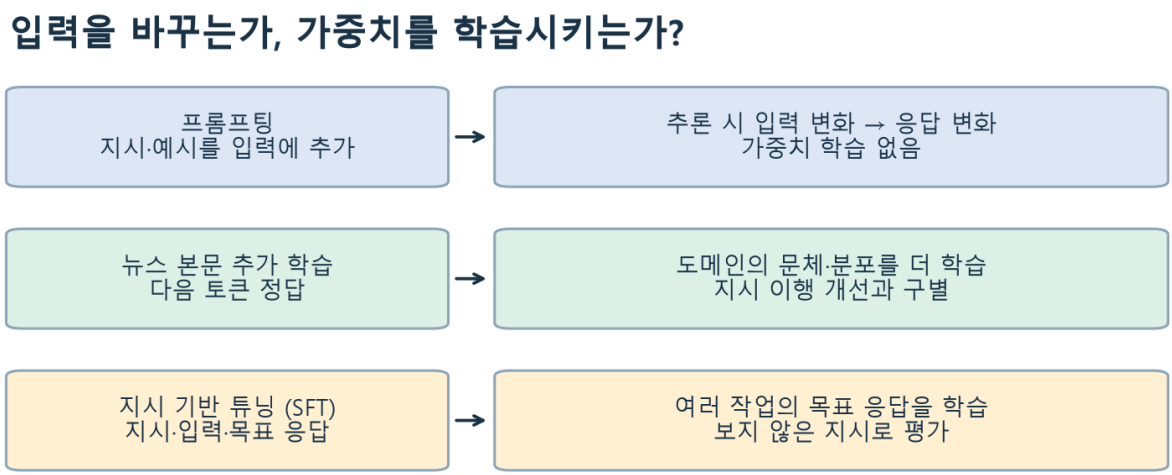



그림 읽기: 프롬프트와 학습 데이터의 내용을 함께 보세요. 뉴스 본문을 다음 토큰으로 학습한 모델과 지시·목표 응답을 학습한 모델은 데이터의 구성과 목적이 다릅니다. 출처: 주교재6.2–6.3을 연결해 새로 설계한 수업 도식.

## 2. 같은 뉴스를 지시 학습 예제로 바꾸기

다음은 수업에서 직접 만든 예문입니다. AG News 원문을 복제한 것이 아닙니다.

| 필드 | 설명용 예 |
|---|---|
| instruction | Rewrite the sentence as a short headline. |
| input | A company opened a new office in the city. |

| 필드 | 설명용 예 |
|---|---|
| response | Company Opens New City Office |

`instruction`은 해야 할 작업, `input`은 작업할 자료, `response`는 학습에서 보여 줄 목표 응답입니다. 필드의 이름만 갖추었다고 좋은 데이터가 되는 것은 아닙니다. 지시가 애매하거나 목표 응답이 틀리면 그 데이터도 문제를 가집니다.

이를 하나의 문자열로 합치는 형식은 여러 가지가 있습니다. `### Instruction`, `### Input`, `### Response` 구분자는 이번 수업의 설명용 직렬화 형식입니다. 모든 실제 모델이 반드시 이 문자열을 사용한다는 뜻은 아닙니다. chat 모델은 자기 토크나이저의 chat template을 사용할 수 있습니다.

이번 버튼은 예제를 구성하고 구조를 보여 줍니다. **가중치를 갱신하는 SFT 학습은 이 활동에서 수행하지 않습니다.** 데이터를 만들었다는 기록, 실제 학습했다는 기록, 지시를 잘 따른다는 평가를 따로 남깁니다.

## 3. 학습목표로 연결하기

지시를 $I$, 입력을 $x$, 목표 응답 토큰을 $r_1, \ldots, r_T$라 할 때, 목표 응답을 학습하는 손실은 다음과 같이 생각할 수 있습니다.

$$\mathcal{L}_{\mathrm{SFT}} = -\frac{1}{T}\sum_{t=1}^{T} \log p_\theta(r_t \mid I, x, r_{<t}).$$

이 식은 응답 토큰에만 손실을 적용하는 설명입니다. 실제 도구의 데이터 수집기·템플릿·loss mask 설정을 확인해야 합니다. 지시·입력까지 포함한 모든 토큰에 손실을 적용하는 구성도 있으므로 "SFT이면 항상 같은 위치를 학습한다"고 단정하지 않습니다.

문장 분류의 네 클래스 정답, 뉴스 본문의 다음 토큰 정답, 지시·입력에 대한 응답 정답을 다시 비교해 보세요. 모델의 인터페이스 이름보다 **입력·정답·손실·갱신 대상**을 확인하는 것이 중요합니다.

## 4. 지시 데이터의 품질을 점검하기

| 점검할 것 | 살펴볼 사례 |
|---|---|
| 지시의 명확성 | "좋게 고쳐 줘"보다 목적·길이·출력 형식이 드러나는가? |
| 목표 응답의 정확성 | 뉴스 주제와 다른 클래스 이름을 정답으로 넣지 않았는가? |
| 내용과 형식의 일관성 | 한 예제는 클래스만, 다른 예제는 장문 해설을 요구하는 이유가 있는가? |
| 중복·평가 누출 | 같은 뉴스의 다른 지시 버전이 train과 test에 동시에 들어갔는가? |
| 작업의 다양성과 균형 | 한 종류의 문장이나 쉬운 작업에만 치우치지 않았는가? |
| 평가 범위 | 학습 문장의 재현과 새로운 지시의 수행을 구별하는가? |

같은 뉴스에 지시문만 다르게 붙였다고 독립적인 평가 문서가 되지는 않습니다. **원문 문서 단위로 먼저 분리하고**, 각 분할 안에서 지시 변형을 만드는 것이 안전한 출발점입니다. 원문의 바꿔 쓰기나 매우 비슷한 사례가 섞이는 문제는 별도 점검이 필요합니다.

**B5 · 필수 · 설계하고 설명하기:** 새로운 뉴스 예문으로 지시·입력·목표 응답을 작성하고, 학습에 쓰지 않을 평가 사례도 만드세요. 목표는 형식화와 실제 학습을 구별하고 평가 누출을 찾는 것입니다. 힌트: 지시문만 바꾸지 말고 원문 문서가 겹치는지도 확인하세요. 자기 점검: 응답이 타당한 이유와 새로운 사례에서 평가할 기준을 적었나요?

## 5. 여러 작업의 지시를 섞는 이유

교재의 그림6-3은 다양한 입력 지시와 서로 다른 형태의 응답을 하나의 언어 모델에 연결합니다. 분류·짧은 답변·변환 등 여러 작업을 지시 형식으로 학습하면 작업의 관계를 활용할 수 있습니다. 다만 몇 개 예제를 작성했다고 새로운 작업에 일반화한다고 결론 내릴 수는 없습니다.

Flan 연구는 여러 작업의 지시 데이터를 학습에 사용하고, 학습에 포함하지 않은 과제에서 성능을 평가하는 설계를 제시합니다. 이번 수업에서는 대규모 학습을 재현하는 대신 **학습 예제와 전이 평가 사례를 구별하는 원리**를 가져옵니다.

평가에는 요청한 작업을 했는지, 내용이 맞는지, 요구한 형식을 지켰는지, 학습하지 않은 표현에서도 동작하는지를 포함합니다. 모델이 출력한 문자열이 자연스럽다는 이유만으로 모든 항목을 통과시켜서는 안 됩니다.

## 6. 어댑터와 SFT는 서로 배타적인가

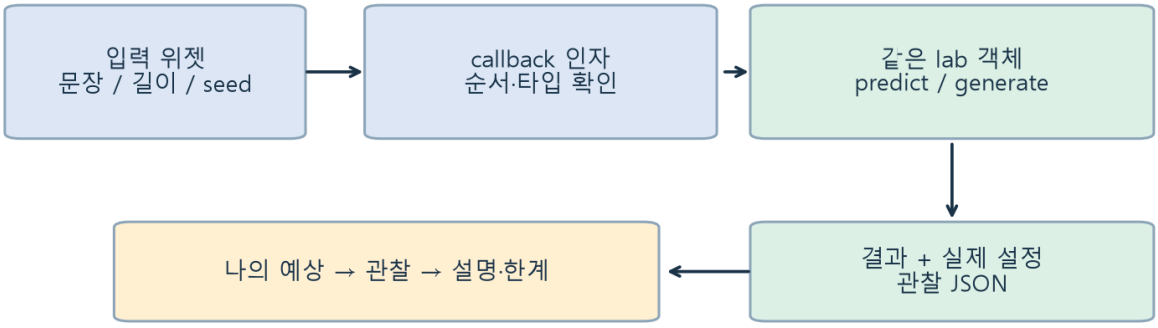
어댑터는 주로 **어떤 파라미터를 추가·갱신하는가**에 관한 접근입니다. SFT는 **어떤 입력·정답으로 어떤 목적을 학습하는가**에 관한 접근입니다. 지시 데이터를 이용한 학습을 전체 파인튜닝으로 할 수도 있고, 적은 수의 파라미터를 갱신하는 방식과 조합할 수도 있습니다.

따라서 프롬프팅·SFT·어댑터를 모두 같은 기준의 선택지로 단순 나열하지 않습니다. 이번 필수 범위는6.3까지이며6.4 이후의 LoRA·양자화 구현은 다음 범위에서 다룹니다.

선택 활동으로 base와 Instruct 모델을 비교할 때도 모델 카드의 학습 이력을 확인합니다. 공개 모델 쌍은 SFT 이외의 학습·선호도 최적화·데이터 변경도 다를 수 있으므로 출력 차이를 SFT 하나의 인과 효과로 단정하지 않습니다.

## 7. 마지막 웹앱에서 방법론을 설명하기

### 노트북의 모델을 웹앱 입력·출력에 연결



앱은 노트북에서 준비한 동일한 모델 객체를 재사용합니다. 버튼을 누를 때마다 다시 모델을 내려받거나 새 학습을 시작하지 않습니다. 입력 위젯 순서와 함수 인자 순서, 반환값과 출력 위젯 순서를 맞춥니다.

**B6 · 필수 · 직접 연결하기:** 자신의 예상·관찰·설명을 정리하는 함수를 작성하여 앱의 입력 세 개와 출력에 연결하세요. 연결한 뒤 새로운 예문을 넣고 한 조건만 바꿉니다. 힌트: 먼저 함수를 직접 호출해 반환값을 확인합니다. 자기 점검: 앱이 실행된 것과 자신이 입력·결과 관계를 설명한 것을 각각 확인했나요?

실험 설명에는 사용한 모델·학습 상태·입력·바꾼 조건·관찰·판단의 한계를 남깁니다. 공유 링크는 런타임 동안 유지되므로 링크가 종료되면 노트북에서 다시 실행해 새 주소를 확인합니다.

## 출처와 추가 설명

- 주교재: 『핸즈온 생성형 AI』 6.3,268–273쪽 상부. 지시 데이터 예문·누출 사례·평가 기준·앱 활동은 수업용으로 새로 작성했습니다.
- Scaling Instruction-Finetuned Language Models
- SmolLM Instruct 학습 이력과 사용 예
- 수업 API 정의와 데이터 형식화의 실행 범위